

The Cost of Zero-Trust for Smart-Lock Systems

CHRISTOPHE-MOKILI ANDUNDA (2210781001@fh-burgenland.at) and BENJAMIN HUANG WEIXIANG (2210781004@fh-burgenland.at)

ABSTRACT: Digitalization has increasingly transformed physical infrastructures into connected cyber-physical systems (CPS). Buildings, shared facilities, rental spaces, and workplaces are no longer managed only through manual processes, but through software platforms, cloud services, Application Programming Interfaces (APIs), and connected devices. In this context, digital access control has become especially relevant, because decisions about physical entry are increasingly handled by cloud-based systems.

Simple mechanisms such as static keypad codes are easy to deploy and use, but they can be shared, leaked, or reused indefinitely without any check at the moment of entry. Stronger access control mechanisms address this by re-verifying a request at the moment of access. This leads to a security-cost trade-off: higher assurance can reduce the risk of unauthorized access, but it also increases infrastructure resource usage, operational complexity, and cloud cost.

This paper examines this trade-off using a microservice-based booking platform for music studios deployed on Amazon Web Services (AWS). The system consists of a User Service, Studio Service, Booking Service, and Access Service. Three access control scenarios with increasing security levels are implemented and compared. The first scenario generates the access code at booking time, and the code is then valid on its own with no further check at the door. The second scenario instead generates the code only at the door, after the system validates the booking code against the current time window. The third scenario builds upon the second by additionally requiring a face verification step before the code is generated, confirming that the person present at the door matches the booking.

To quantify this trade-off, the evaluation measures cost per access attempt, average Central Processing Unit (CPU) utilization, and average memory utilization. These metrics are selected because they show how additional verification logic affects the infrastructure footprint of the Access Service, which contains the scenario-specific access control logic. By comparing the three scenarios under the same deployment conditions, this work provides a practical basis for analyzing the operational cost of stronger access control in smart-lock-based CPS.

1 Introduction

Cyber-Physical Systems (CPS) and the so-called Internet of Things (IoT)-based access control solutions are increasingly relevant in domains such as smart homes, short-term rentals, shared facilities, and building automation. In particular, smart locks enable flexible, cloud-based management of access permissions and support automated processes such as check-in and check-out. These systems consist of multiple interacting components, including user interfaces, cloud services, databases, Application Programming Interfaces (APIs), and physical devices.

The adoption of smart locks has grown rapidly in recent years, with the global market expected to reach over US\$8 billion by 2030 [5]. Platforms such as Airbnb rely on smart locks to enable automated check-in and remote access management, eliminating the need for physical key exchange. As a result, access control is no longer purely physical but is increasingly mediated through software systems, APIs, and cloud services. This increases system complexity and introduces additional attack surfaces.

This increase in potential attack surfaces is accompanied by growing security concerns. Prior research has shown that Bluetooth-based smart locks can be vulnerable to eavesdropping and man-in-the-middle attacks [15], while replay attacks can enable unauthorized unlocking by reusing captured communication [4]. Additionally, flaws in smart home platforms have shown that attackers can inject credentials or unlock doors via compromised cloud services [7]. These findings highlight that smart lock systems can introduce vulnerabilities across device, communication, and cloud layers.

This leads to a fundamental trade-off between usability, security, and infrastructure cost. Simple mechanisms such as static keypad codes are easy to use, but they are vulnerable to sharing, leakage, and unauthorized reuse. In contrast, stronger access control mechanisms, such as contextual validation or biometric verification, can improve protection but may also increase operational cost, Central Processing Unit (CPU) usage, memory usage, and system complexity. Similar to prior work on the measurable cost of security mechanisms, such as the cost introduced by the Hypertext Transfer Protocol Secure (HTTPS) [14], enhanced access control in smart-lock-based systems can also create measurable infrastructure overhead.

In this position paper, we investigate this trade-off in the context of bcube, a cloud-based booking platform for music studios. The system follows a microservice architecture and consists of a User Service, Studio Service, Booking Service, and Access Service. A Nuki Smart Lock is used as the physical access mechanism, while the cloud-based Access Service controls when a valid access code is generated or released.

We compare three access control scenarios with increasing security levels. The first scenario uses direct keypad access after a completed booking. The second scenario requires booking-code validation before a temporary keypad access code is generated. The third scenario additionally includes face verification before the access code is generated. The goal is to evaluate how these different security levels affect operational cost and infrastructure resource usage.

The evaluation focuses on three concrete metrics: cost per access attempt, average CPU utilization, and average memory utilization. These metrics are measured mainly for the Access Service, because it contains the scenario-specific access logic and is expected to show the clearest differences between the implemented security levels.

The remainder of this paper is structured as follows: Section II reviews related work, Section III presents the methodology and experimental setup, and Section IV concludes the paper.

2 Related Work

The cost of security mechanisms has been widely studied, particularly in the context of network protocols. A key reference is [14], where the authors evaluated the cost of the "S" in HTTPS, demonstrating that adding security through TLS introduces additional data overhead and computational cost. This work provides a foundation for analyzing security not only as a functional property, but also as a measurable source of resource consumption.

In the context of Cyber-Physical Systems (CPS), Ivkić et al. [10–12] propose a series of security cost modeling frameworks that highlight the importance of evaluating security mechanisms alongside their resource and operational overhead. First, Ivkić et al. [10] introduced the concept of modeling security costs in terms of resource consumption, showing that such mechanisms can significantly affect constrained environments. Next, they extended this by proposing a runtime framework [11], demonstrating that security overhead varies dynamically with system state and workload. Finally, they presented a more comprehensive model integrating computational, communication, and operational costs [12], emphasizing the need for holistic evaluation of security decisions.

Recent work has identified security vulnerabilities in smart lock systems across several components. Ho et al. [9] analyze five commercial smart locks and show that weaknesses can arise not only from implementation bugs, but also from system design, revocation handling, access logging, and automatic unlocking interaction models. Caballero-Gil et al. [2] and Turk [17] report weaknesses in Bluetooth communication, including replay and interception attacks, while Diao et al. [4] identify vulnerabilities in device protocols that allow unauthorized access persistence and manipulation of access rights. Veijalainen and Karlsson [18] complement these works with a threat-model-driven penetration test of a smart door lock system, deriving concrete tests from reconnaissance and STRIDE/DREAD-based threat modeling to analyze app-server communication, replay behavior, gateway exposure, and key-tag access. Together, these works show that vulnerabilities in smart lock systems can already arise at the device and communication layer, before any cloud-side access control logic is even involved.

Beyond the device and communication layer, vulnerabilities have also been identified in the companion mobile applications that control smart locks. Allen et al. [1] highlight security flaws in companion mobile applications, such as exposure of API endpoints and authentication mechanisms. Complementing these findings, Wang et al. [19] propose a scalable methodology to detect such issues through the analysis of mobile companion applications, showing that insecure design patterns can propagate across multiple devices due to code reuse. Together with the device-level findings above, these results demonstrate that vulnerabilities can occur at different layers of smart lock systems, motivating the need for stronger access control mechanisms.

User-oriented research further shows that smart lock security cannot be evaluated only from a technical perspective. Hazazi and Shehab [8] investigate smart lock automation in smart home environments and show that users perceive such automation in terms of security, convenience, awareness, privacy, and reliability. This is relevant for access-control systems because stronger verification mechanisms must not only improve protection, but also remain understandable and acceptable for users.

Two complementary implementation-oriented studies investigate the integration of smart lock systems with cloud infrastructures. Salvatore [16] proposes a cloud-based orchestration system built on API-based integration with smart lock platforms such as Nuki, where the Web API is used to generate and manage time-limited virtual keys based on booking data. This enables automated access provisioning and reduces manual intervention. However, this approach relies mainly on booking data and does not evaluate additional verification at access time.

In contrast, Zebic [20] examines the integration of a Nuki smart lock into Microsoft Azure from a process-oriented perspective. The work identifies and evaluates the individual steps required for integration, such as configuring authentication parameters, establishing API connections, and managing device-specific settings. By introducing metrics to assess the complexity of these steps, the study shows that integrating smart locks into cloud environments involves multiple configuration tasks that go beyond simple API usage.

Other prototype-based studies illustrate how smart locks can be implemented using embedded devices and cloud services. Maurya et al. [13] implement a cloud-native door lock system using Raspberry Pi, Arduino, Alexa, and Amazon Web Services, including services for image storage, face recognition, messaging, and device communication. Derbali [3] proposes an Internet of Things smart door lock system with temporary PIN codes, Radio Frequency Identification, Bluetooth-based access, remote unlocking, and access records. Zheng et al. [21] present an STM32-based smart door lock with multiple unlocking methods, including RFID, fingerprint recognition, keypad access, remote unlocking, and a vibration alarm. These works demonstrate the technical feasibility of cloud-connected and embedded smart lock systems.

While existing work has extensively analyzed security vulnerabilities, usability concerns, and system integration challenges in smart lock environments, the infrastructure cost associated with stronger access control mechanisms has not been studied as extensively. Additional verification steps, such as contextual validation or biometric verification, can improve access control but also introduce more complex system interactions, additional cloud service usage, and increased resource consumption.

To address this gap, this position paper focuses on the cost and infrastructure impact of stronger access control mechanisms in a cloud-managed smart lock system. By comparing three access control scenarios with increasing security levels, ranging from direct keypad access to booking-code validation and face verification, this work evaluates how stronger access control affects operational cost, CPU utilization, and memory utilization in a realistic cloud-based booking system.

3 Methodology

This chapter describes the methodological design used to evaluate the cost of stronger access control in a smart-lock-based booking system. The evaluation follows a comparative approach: the same booking and access use case is implemented with three different access control scenarios, each representing a different security level. The scenarios are then compared using cloud cost and infrastructure resource metrics.

3.1 High-Level Use Case

The use case considered in this work is a booking-based access control process for music studios. A user books a studio for a specific time window and should only be able to enter the room during this valid booking period. The physical entry point is protected by a Nuki Smart Lock and keypad, while the access decision is controlled by a cloud-based backend system.

At a high level, the process consists of four steps. First, a user creates a booking for a studio. Second, the system stores the booking and associates it with a valid access period. Third, the user attempts to access the studio at the door. Fourth, the system either grants or denies access depending on the selected access control scenario. This is illustrated in Figure 1.

This leads to a security-cost trade-off: higher assurance can reduce the risk of unauthorized access, but it also increases infrastructure resource usage, operational complexity, and cloud cost.

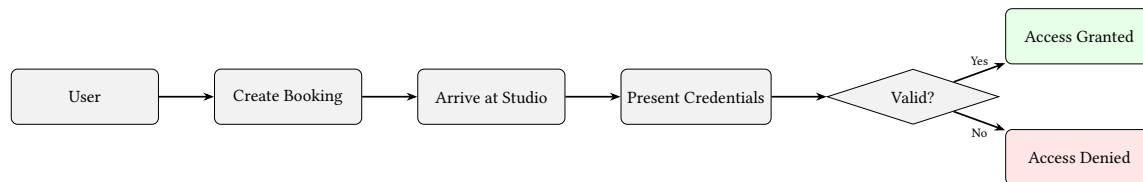


Fig. 1. High-level booking and access control use case.

3.2 System Architecture and Roles

The prototype is implemented as a microservice application consisting of four Spring Boot 3.5.6 services deployed on AWS ECS Fargate and exposed via an Application Load Balancer (ALB) with path-based routing rules. All services persist data in dedicated Amazon RDS PostgreSQL 17 databases on a db.t3.micro instance. Table 1 summarizes the main roles of the system components.

Table 1. System roles and responsibilities

ID	Component	Responsibility
R1	User Service	Manages authentication and user data. Issues JWT tokens signed with a shared HMAC-SHA256 secret and validates incoming Bearer tokens using Spring Security’s JWT decoder.
R2	Studio Service	Manages studio entities, availability, and Nuki smartlock associations.
R3	Booking Service	Creates and stores bookings, including the booked studio, user, and valid time window. Upon booking creation, it synchronously calls the Access Service to initialize the access permission.
R4	Access Service	Implements the scenario-specific access control logic. This is the primary measurement subject in this evaluation.
R5	Nuki Smart Lock	Provides the physical access mechanism. The lock executes time-limited keypad codes registered by the Access Service via the Nuki Web API.

The Access Service (R4) is deployed as a single ECS Fargate task with 0.25 vCPU and 512 MB memory. The active security scenario is selected at startup via the `FA_LEVEL` environment variable, which activates the corresponding implementation through Spring Boot’s `@ConditionalOnProperty` mechanism. All three implementations share a common abstract base class and differ only in the access-time flow.

For face verification (Cases 2 and 3), the system relies on a pre-built Amazon Rekognition face collection. User face enrollment is handled by a separate service in a distinct repository, implementing the following pipeline: a user photo is uploaded to Amazon S3, which triggers an AWS Lambda function that invokes the Rekognition `IndexFaces` API and stores the resulting `FaceId` and user identity in Amazon DynamoDB. This enrollment pipeline runs once per user and is independent of the per-access-attempt flow evaluated in this work.

3.3 Access Control Scenarios

Three access control scenarios are implemented and compared. They represent increasing security levels and differ in how much verification is performed at the moment of access.

Case 1: Single-Factor Access (1FA). At booking creation, the Access Service generates a six-digit PIN, encrypts and stores it in PostgreSQL, and immediately registers a time-limited keypad code on the Nuki lock via the Nuki Web API. At access time, the user enters this PIN directly at the Nuki keypad without any further backend interaction. The sole authentication factor is knowledge of the booking code. No AWS managed services are invoked per access attempt.

Case 2: Two-Factor Access (2FA). At booking creation, the Access Service stores the encrypted PIN in the database without registering a Nuki code. At access time, the following steps are performed: (1) the user submits the booking PIN on a separate dashboard, which the Access Service validates against a stored SHA-256 hash. (2) The user captures a selfie, which the Access Service passes in-memory to the Amazon Rekognition API against the pre-built face collection. (3) Upon successful biometric verification, the Access Service generates a new Nuki code, registers it via the Nuki Web

API, and returns it to the application for display. The two authentication factors are knowledge of the booking code and biometric identity.

Case 3: Three-Factor Access (3FA). Case 3 is identical to Case 2 in all steps except the delivery of the Nuki code. Instead of returning the code to the application for display, the Access Service invokes Amazon SES to send the code to the user's registered email address. The code is not shown in the application. The three authentication factors are knowledge of the booking code, biometric identity, and possession of the registered email account.

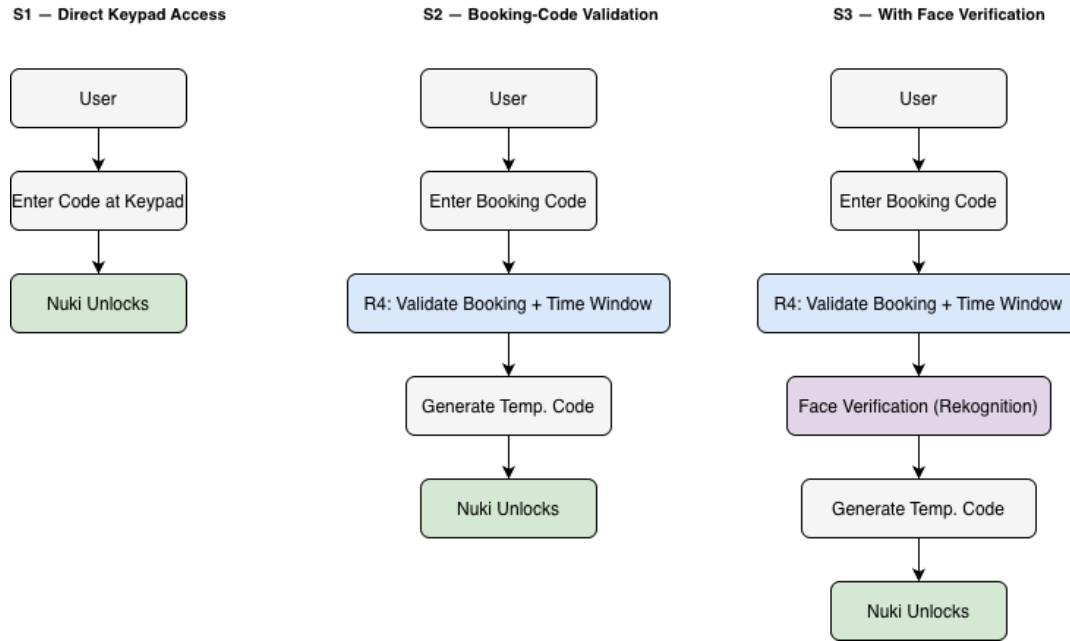


Fig. 2. Comparison of the three access control scenarios with mapped evaluation metrics.

3.4 Evaluation Metrics

The evaluation uses three metrics to quantify the cost of stronger access control. The metrics focus on the Access Service because it contains the scenario-specific access logic. The User Service, Studio Service, and Booking Service behave identically across all scenarios and are excluded from metric collection accordingly.

- **M1: Cost per Access Attempt.** The average variable AWS service cost per access attempt in each scenario. Calculated by dividing the total scenario-related variable cloud cost by the number of executed access attempts. Fixed infrastructure costs (ECS Fargate, RDS, VPC, ALB) are excluded as they are invariant across scenarios.
- **M2: Average CPU Utilization.** The average CPU utilization of the Access Service during the test window, reported as a percentage of the allocated 0.25 vCPU, collected via Amazon CloudWatch Container Insights at one-minute granularity.
- **M3: Average Memory Utilization.** The average memory utilization of the Access Service during the test window, reported as a percentage of the allocated 512 MB, collected via Amazon CloudWatch Container Insights at one-minute granularity.

The cost per access attempt (M1) is calculated as follows:

$$M1 = \frac{\text{Total Scenario-Related Variable AWS Cost}}{\text{Number of Access Attempts}} \quad (1)$$

3.5 Measurement and Data Collection

Variable cost data is derived from AWS public pricing as of June 2026, applied to the exact number of external service invocations per access attempt. At access time, Case 2 invokes one Rekognition SearchFacesByImage call (\$0.001 per image). Case 3 additionally invokes one Amazon SES send (\$0.0001 per message). Case 1 invokes no AWS managed services at access time. The face enrollment pipeline (S3, Lambda, IndexFaces, DynamoDB) is a one-time per-user operation and is excluded from per-attempt cost calculations. The Nuki Web API is billed externally to AWS and is also excluded.

Fixed infrastructure costs (ECS Fargate, RDS, VPC, ALB) are treated as a shared baseline invariant across all three scenarios and are reported separately for reference.

CPU and memory utilization metrics are collected via Amazon CloudWatch Container Insights for the Access Service ECS task, using one-minute average statistics. Because all three scenarios run on identical task configurations (0.25 vCPU, 512 MB), the reported percentages are directly comparable without normalization.

3.6 Test Execution

Each scenario is executed under identical infrastructure conditions. The same ECS task configuration, RDS instance class, ALB listener rules, and CloudWatch monitoring setup are applied consistently. No auto-scaling policies are active during the test window.

For each scenario, 20 bookings and 20 corresponding access attempts are executed sequentially via the application under controlled conditions. All three runs are conducted on 2026-06-18 with the FA_LEVEL environment variable set to 1, 2, and 3 for Case 1, Case 2, and Case 3, followed by an ECS force redeployment between scenarios.

After execution, data is grouped by scenario and evaluated against M1, M2, and M3.

4 Results and Analysis

4.1 Overview of Results

This section presents the measured results for all three access control scenarios across the three evaluation metrics: cost per access attempt (M1), average CPU utilization (M2), and average memory utilization (M3). All scenarios were executed with 20 bookings and 20 corresponding access attempts under identical infrastructure conditions.

Table 2 provides a summary of all measured values. Cost figures include the fixed monthly infrastructure baseline of \$4.43 amortized over 1,000 access attempts, making the comparison practically meaningful.

Table 2. Measured results per scenario (cost at 1,000 monthly access attempts)

Case	Security Level	M1 (Total/Month)	M2 (Avg. CPU)	M3 (Avg. Mem.)
Case 1	Low	\$4.43	3.1%	41.2%
Case 2	Medium	\$5.43	3.4%	41.8%
Case 3	High	\$5.53	3.3%	42.1%

4.2 Cost per Access Attempt (M1)

The total monthly cost of operating the system consists of two components: a fixed infrastructure baseline that is invariant across all scenarios, and a variable per-attempt cost that depends on the managed AWS services invoked during each access event.

Fixed Infrastructure Baseline. The shared ECS Fargate task, RDS db.t3.micro instance, VPC, and ALB components incur a combined fixed cost of \$4.43 per month regardless of which access control scenario is active or how many access attempts are processed. This baseline represents the minimum operational cost of running the system and is identical for all three cases.

Variable Cost per Attempt. Case 1 invokes no external AWS services at access time and therefore adds no variable cost on top of the baseline. Case 2 introduces a single call to Amazon Rekognition SearchFacesByImage per access attempt, priced at \$0.001 per image (AWS pricing, June 2026). Case 3 is identical to Case 2 but additionally dispatches the Nuki code via Amazon SES, priced at \$0.0001 per message, bringing its per-attempt variable cost to \$0.0011.

Table 3 breaks down the variable cost components per scenario.

Table 3. Variable cost breakdown per access attempt (AWS pricing, June 2026)

Case	Service Invocation	Unit Price (USD)	Variable Total
Case 1	None	–	\$0.0000
Case 2	Rekognition SearchFacesByImage	\$0.0010	\$0.0010
Case 3	Rekognition SearchFacesByImage Amazon SES (1 message)	\$0.0010 \$0.0001	\$0.0011

Total Monthly Cost. Table 4 combines the fixed baseline with the projected variable cost at three usage volumes to illustrate how the cost structure shifts with scale.

Table 4. Total projected monthly cost including fixed infrastructure baseline

Case	Fixed	100 att.	1,000 att.	10,000 att.
Case 1	\$4.43	\$4.43	\$4.43	\$4.43
Case 2	\$4.43	\$4.53	\$5.43	\$14.43
Case 3	\$4.43	\$4.54	\$5.53	\$15.43

At low usage (100 attempts/month), the fixed baseline dominates and all three scenarios cost nearly the same (\$4.43 to \$4.54). At 1,000 attempts the security surcharge of Case 3 over Case 1 is \$1.10 (+24.8%). At 10,000 attempts the variable cost of Case 3 (\$11.00) exceeds the fixed infrastructure cost by a factor of approximately 2.5, making the choice of security scenario the primary cost driver.

4.3 CPU and Memory Utilization (M2, M3)

Both CPU and memory utilization of the Access Service remained at comparable levels across all three scenarios, as shown in Table 2. The observed variation (3.1% to 3.4% for CPU, 41.2% to 42.1% for memory) falls within the margin of measurement noise inherent to one-minute CloudWatch sampling and does not represent a meaningful operational difference.

This outcome is consistent with the architectural design of the system. The Access Service is I/O-bound: its main role at access time is to invoke external managed services (Rekognition, SES) and await their responses. The computationally intensive operations, namely face recognition inference and email relay, are executed on AWS-managed infrastructure and do not contribute to the Access Service’s CPU or memory footprint. The additional Rekognition and SES calls introduced by Cases 2 and 3 manifest as network round-trips and API latency, not as additional local compute load.

The memory footprint of approximately 41 to 42% across all scenarios corresponds to the baseline JVM heap usage of the Spring Boot application and the in-memory image buffer used during face verification. Since the selfie image is passed directly to the Rekognition API and not persisted, its transient allocation does not accumulate over repeated attempts.

Consequently, M2 and M3 are not suitable discriminating metrics for the scenarios evaluated in this work. The relevant resource impact of higher security levels is captured entirely by M1.

4.4 Comparison of Security Levels

Figure 3 illustrates the total monthly cost at 1,000 access attempts across the three scenarios, combining the fixed baseline with scenario-specific variable costs. The step from Case 1 to Case 2 represents the dominant cost increment, driven entirely by the Rekognition integration. The additional surcharge from Case 2 to Case 3 (SES delivery) is marginal in comparison.

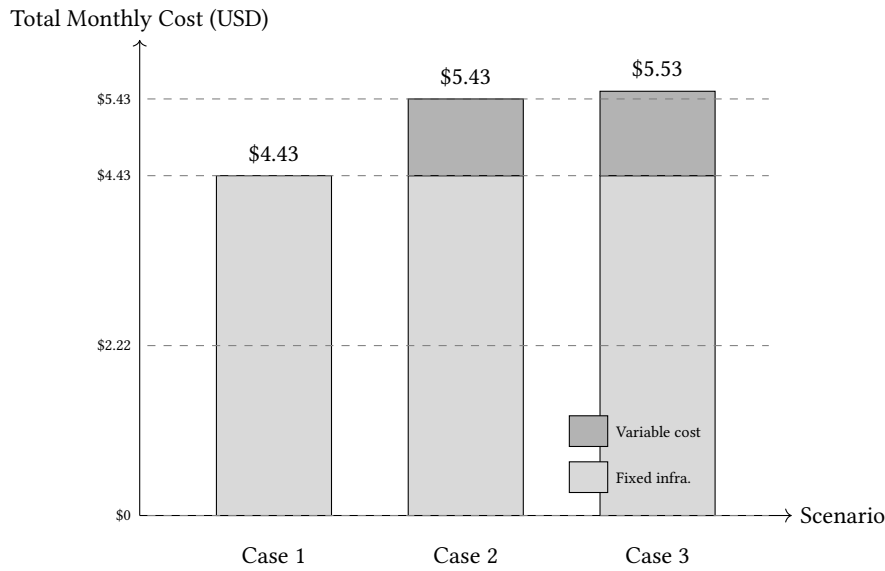


Fig. 3. Total monthly cost by scenario at 1,000 access attempts (fixed + variable).

4.5 Interpretation of Findings

The results confirm that the fixed infrastructure baseline (\$4.43/month) is the dominant cost component at low to moderate access volumes. At 1,000 monthly attempts, it accounts for 80% of the total cost in Case 3. The choice of security scenario therefore has limited practical cost impact until access volumes reach several thousand attempts per month.

When variable costs become significant at higher volumes, the primary driver is the integration of Amazon Rekognition for biometric face verification (Case 2 and 3), not the out-of-band delivery channel (Case 3 only). Rekognition accounts for \$0.001 of the \$0.0011 per-attempt cost in Case 3, approximately 91% of the total variable cost of the highest-security scenario. The SES surcharge of \$0.0001 per attempt is negligible by comparison.

The absence of measurable differences in CPU and memory utilization across scenarios reflects the delegation model inherent to cloud-native architectures: managed services externalize computational work, making on-task resource metrics an unreliable proxy for security-related operational cost. The true cost of higher security assurance is best captured through API billing metrics rather than infrastructure utilization metrics.

5 Limitations and Future Work

The scenario design in this work is informed by the common distinction between authentication factors — something the user *knows*, something the user *has*, and something the user *is* — as defined in NIST SP 800-63B [6]. However, the three scenarios do not constitute single-, two-, and three-factor authentication in the formal sense of this standard. NIST SP 800-63B defines possession-based factors as physical or cryptographic authenticators, such as hardware tokens or cryptographic keys bound to a device. A booking code does not meet this definition because it is a knowledge-based secret that can be copied, shared, or forwarded. S2 therefore does not introduce a second authentication factor; instead, its security gain comes from contextual enforcement at the moment of entry rather than from a distinct possession factor.

A second limitation concerns the scale of the evaluation. Each scenario was executed with 20 bookings and 20 access attempts under controlled and identical infrastructure conditions. While this allows for a direct comparison between scenarios, it does not reflect production-scale traffic patterns, concurrent access attempts, or variable load conditions. The observed cost and resource usage values should therefore be interpreted as indicative rather than representative of real-world deployments.

Finally, the evaluation focuses exclusively on the Access Service (R4) as the measurement point. Costs and resource usage associated with supporting services such as Amazon Rekognition, the Nuki Web API, or the database layer are included in the cost metric M1 but are not individually isolated. This means that the contribution of each external dependency to the overall cost difference between scenarios cannot be precisely attributed from the current measurements alone.

6 Conclusion

Access control in IoT-enabled physical spaces sits at the intersection of security engineering and cloud economics. As smart lock systems move from simple PIN-based mechanisms to biometric and multi-factor verification, the cost implications of that transition remain underexplored in practice.

This work addresses the question of how much stronger access control actually costs in a deployed cloud environment. The gap between theoretical security gains and their operational price is rarely quantified in concrete, reproducible terms.

To answer this, a prototype booking and access system was implemented as four Spring Boot microservices on AWS ECS Fargate and evaluated across three scenarios: single-factor PIN access (1FA), two-factor biometric verification via Amazon Rekognition (2FA), and three-factor verification with out-of-band code delivery via Amazon SES (3FA).

The central finding is that the cost of stronger access control is dominated by the integration of managed AI inference services, not by compute or infrastructure overhead. Moving from 1FA to 3FA adds \$0.0011 per access attempt in variable costs, with Rekognition accounting for 91% of that figure. CPU and memory utilization of the Access Service remain indistinguishable across all three scenarios, as the computationally intensive work is offloaded to AWS-managed infrastructure. At moderate usage volumes, the fixed infrastructure baseline of \$4.43 per month outweighs the security surcharge by a wide margin.

Future work should investigate the cost behavior at higher access volumes, evaluate caching strategies to reduce repeated Rekognition invocations for frequent users, and extend the comparison to additional physical access scenarios beyond music studio bookings.

References

- [1] Joseph Allen, Lo'ai Tawalbeh, Fadi Muheidat, Rami Alkhawaldeh, and Yaser Jararweh. 2024. Security Evaluation of Companion Android Applications in IoT: The Case of Smart Security Devices. *Sensors* 24, 17 (2024), 5465. <https://www.mdpi.com/1424-8220/24/17/5465>
- [2] Cándido Caballero-Gil, Rafael Álvarez, Candelaria Hernández-Goya, and Jezabel Molina-Gil. 2024. Research on smart-locks cybersecurity and vulnerabilities. *Wireless Networks* 30 (2024), 5905–5917.
- [3] Morched Derbali. 2023. Toward Secure Door Lock System: Development IoT Smart Door Lock Device. Preprint. Posted: 2023-04-03. doi:10.22541/au.168055385.54003954/v1
- [4] Chengsong Diao, Danielle Dang, Sierra Lira, Angela Tsai, Miro Haller, and Nadia Heninger. 2025. No Key, No Problem: Vulnerabilities in Master Lock Smart Locks. In *Proceedings of the 19th USENIX Conference on Offensive Technologies (WOOT '25)*. USENIX Association, 69–85. <https://dl.acm.org/doi/10.5555/3769714.3769719>
- [5] Grand View Research. 2023. Smart Lock Market Size, Share & Trends Analysis Report. <https://www.grandviewresearch.com/industry-analysis/smart-lock-market>. Accessed: 2026-04-27.
- [6] Paul A. Grassi, Michael E. Garcia, and James L. Fenton. 2017. *NIST Special Publication 800-63B: Digital Identity Guidelines – Authentication and Lifecycle Management*. Technical Report. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-63b
- [7] Andy Greenberg. 2016. Flaws in Samsung's Smart Home Let Hackers Unlock Doors. <https://www.wired.com/2016/05/flaws-samsungs-smart-home-let-hackers-unlock-doors-set-off-fire-alarms/>. Accessed: 2026-04-27.
- [8] Hussein Hazazi and Mohamed Shehab. 2024. Exploring End Users' Perceptions of Smart Lock Automation Within the Smart Home Environment. In *Proceedings of the 2024 European Symposium on Usable Security (EuroUSEC 2024)*. ACM. doi:10.1145/3688459.3688480
- [9] Grant Ho, Derek Leung, Pratyush Mishra, Ashkan Hosseini, Dawn Song, and David Wagner. 2016. Smart Locks: Lessons for Securing Commodity Internet of Things Devices. In *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security (ASIA CCS '16)*. ACM, 461–472. doi:10.1145/2897845.2897886
- [10] Igor Ivkić, Andreas Mauthe, and Markus Tauber. 2019. Towards a security cost model for cyber-physical systems. In *2019 16th IEEE Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 1–7.
- [11] Igor Ivkić, Harald Pichler, Mario Zsilak, Andreas Mauthe, and Markus Tauber. 2019. A framework for measuring the costs of security at runtime. *arXiv preprint arXiv:1905.11180* (2019).
- [12] Igor Ivkić, Patrizia Sailer, Antonios Gougliadis, Andreas Mauthe, and Markus Tauber. 2022. A security cost modelling framework for cyber-physical systems. *ACM Transactions on Internet Technology (TOIT)* 22, 2 (2022), 1–31.
- [13] Rishabh Maurya, Kushagra Jaiswal, Prankur Kohli, Shikar Kumar, and Abhilasha Varshney. 2020. Design and Implementation of Cloud Native Door Lock System. *International Journal of Engineering Research & Technology* 9, 5 (2020), 985–987.
- [14] David Naylor, Alessandro Finamore, Ilias Leontiadis, Yan Grunenberger, Marco Mellia, Maurizio Munafò, Konstantina Papagiannaki, and Peter Steenkiste. 2014. The Cost of the "S" in HTTPS. In *CoNEXT*. 133–140.
- [15] Nordic Semiconductor. 2018. Wireless Smart Lock Makers React to Security Scares. <https://www.nordicsemi.com/Nordic-news/2018/08/Wireless-smart-lock-makers-react-to-security-scares>. Accessed: 2026-04-27.
- [16] Nicola Salvatore. 2022. *Study and design of an application for door smart-locks and smart-keys*. Master's thesis. University of Padova.
- [17] K. Turk. 2023. *Locked into stupidity: A smart lock security analysis*. Master's thesis. University of Cambridge.
- [18] Sebastian Veijalainen and Tommy Noreng Karlsson. 2021. *Evaluating the Security of a Smart Door Lock System*. KTH Thesis Report. KTH Royal Institute of Technology, Stockholm, Sweden.
- [19] Xueqiang Wang, Chengyu Zhang, et al. 2019. Looking from the Mirror: Evaluating IoT Device Security through Mobile Companion Apps. In *Proceedings of the 28th USENIX Security Symposium*. 1151–1168.
- [20] Luka Zebic. 2023. *IoT Implementations in Microsoft Azure*. Master's thesis. FH Campus Vienna.
- [21] Zhiguo Zheng, Yujie Wang, Jie Chen, Shuteng Lin, and Meirong Lin. 2024. Design and Implementation of a Smart Door Lock Based on STM32. In *Proceedings of the 2024 8th International Conference on Electronic Information Technology and Computer Engineering (EITCE 2024)*. ACM, 1348–1352. doi:10.1145/3711129.3711355